WEEK 5 – PAYMENT TRANSACTION MANAGEMENT SYSTEM

- **S.Kaviya(E25AI119)**

1.Scenario

The Art Work Shop database includes a Payment Transaction Module to store and manage customer payment details. It records the payment ID, order ID, payment date, payment mode, payment status and transaction amount. This module helps track payments and analyse transactions made for customer orders.

2.Create payment table

```
SQL> CREATE TABLE Payment (Payment_ID NUMBER, Order_ID NUMBER, Payment_Date
DATE, Payment_Mode VARCHAR2(20), Payment_Status VARCHAR2(20), Transaction_Am
ount NUMBER(10,2));

Table created.
```

3.Applying constraints

- **PRIMARY KEY:** Uniquely identifies each payment record using Payment_ID.
- **FOREIGN KEY:** Links Order_ID in the Payment table with Order_ID in the Orders table.
- **NOT NULL:** Ensures that required payment fields cannot be left empty.
- **CHECK:** Ensures that the Transaction_Amount is greater than zero.
- **UNIQUE:** Ensures that each order has only one payment record.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT PK_PAYMENT PRIMARY KEY (Payment_ID);

Table altered.

SQL> ALTER TABLE Payment ADD CONSTRAINT FK_PAYMENT_ORDER FOREIGN KEY (Order_
ID) REFERENCES Orders(Order_ID);

Table altered.

SQL> ALTER TABLE Payment ADD CONSTRAINT UQ_PAYMENT_ORDER UNIQUE (Order_ID);

Table altered.

SQL> ALTER TABLE Payment MODIFY (Order_ID NOT NULL);

Table altered.

SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_PAYMENT_AMOUNT CHECK (Transactio
n_Amount > 0);

Table altered.
```

```
SQL> desc Payment;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PAYMENT_ID                                NOT NULL NUMBER
 ORDER_ID                                  NOT NULL NUMBER
 PAYMENT_DATE                                       DATE
 PAYMENT_MODE                                       VARCHAR2(20)
 PAYMENT_STATUS                                     VARCHAR2(20)
 TRANSACTION_AMOUNT                                 NUMBER(10,2)
```

4.Establishing Relationship

One-to-One (1:1) Relationship: The Orders table is linked to the Payment table using Order_ID as a Foreign Key and UNIQUE constraint, ensuring that each order has only one payment record.

5.Inserting Values to payment table

```
SQL> INSERT INTO Payment VALUES (1, 1001, '20-SEP-26', 'UPI', 'Success', 70000);
1 row created.
SQL> INSERT INTO Payment VALUES (2, 1002, '21-SEP-26', 'Card', 'Success', 45000);
1 row created.
SQL> INSERT INTO Payment VALUES (3, 1003, '22-SEP-26', 'Cash', 'Failed', 3400);
1 row created.
SQL> INSERT INTO Payment VALUES (4, 1004, '23-SEP-26', 'UPI', 'Pending', 75000);
1 row created.
SQL> INSERT INTO Payment VALUES (5, 1006, '25-SEP-26', 'Card', 'Success', 55000);
1 row created.
SQL> INSERT INTO Payment VALUES (6, 1007, '26-SEP-26', 'UPI', 'Pending', 67500);
1 row created.
SQL> INSERT INTO Payment VALUES (7, 1008, '27-SEP-26', 'Card', 'Success', 45000);
1 row created.
SQL> INSERT INTO Payment VALUES (8, 1009, '28-SEP-26', 'Cash', 'Failed', 3400);
1 row created.
SQL> INSERT INTO Payment VALUES (9, 1010, '29-SEP-26', 'UPI', 'Pending', 7500);
1 row created.
```

```
SQL> select * from Payment;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         1       1001 20-SEP-26 UPI                  Success
             70000

         2       1002 21-SEP-26 Card                 Success
             45000

         3       1003 22-SEP-26 Cash                 Failed
              3400


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         4       1004 23-SEP-26 UPI                  Pending
             75000

         5       1006 25-SEP-26 Card                 Success
             55000

         6       1007 26-SEP-26 UPI                  Pending
             67500


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         7       1008 27-SEP-26 Card                 Success
             45000
```

```
         8       1009 28-SEP-26 Cash                 Failed
              3400

         9       1010 29-SEP-26 UPI                  Pending
              7500


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
        10       1011 30-SEP-26 UPI                  Success
             60000


10 rows selected.
```

Update:

```
SQL> UPDATE Payment SET Payment_Status = 'Success' WHERE Payment_ID = 3;

1 row updated.
```

Delete:

```
SQL> DELETE FROM Payment WHERE Payment_ID = 9;

1 row deleted.
```

6. Manage successful and failed payments

Write SQL queries to

- Display all successful payments:

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Success';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         1       1001 20-SEP-26 UPI                  Success
             70000

         2       1002 21-SEP-26 Card                 Success
             45000

         3       1003 22-SEP-26 Cash                 Success
              3400


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         5       1006 25-SEP-26 Card                 Success
             55000

         7       1008 27-SEP-26 Card                 Success
             45000

        10       1011 30-SEP-26 UPI                  Success
             60000


6 rows selected.
```

- Find failed transactions

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         8       1009 28-SEP-26 Cash                 Failed
              3400
```

- Count total successful and failed payments

```
SQL> SELECT Payment_Status, COUNT(*) AS Total_Payments FROM Payment WHERE Payment_Status IN ('Success', 'Failed') GROUP BY Payment_Status;

PAYMENT_STATUS       TOTAL_PAYMENTS
-------------------- --------------
Failed                            1
Success                           6
```

- Update failed payments after retry

```
SQL> UPDATE Payment SET Payment_Status = 'Success' WHERE Payment_Status = 'Failed';

1 row updated.
```

- Identity pending transactions

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Pending';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         4       1004 23-SEP-26 UPI                  Pending
             75000

         6       1007 26-SEP-26 UPI                  Pending
             67500
```

7.Analyse payment methods used by customers

Generate payment analysis report

Report 1: Payment mode analysis

Find :

- Number of transactions using UPI

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM Payment WHERE Payment_Mode = 'UPI';

UPI_TRANSACTIONS
----------------
               4
```

- Number of card payments

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM Payment WHERE Payment_Mode = 'Card';

CARD_PAYMENTS
-------------
            3
```

- most preferred Payment method

```
SQL> SELECT Payment_Mode, TOTAL
  2  FROM (
  3       SELECT Payment_Mode, COUNT(*) AS TOTAL
  4       FROM Payment
  5       GROUP BY Payment_Mode
  6       ORDER BY COUNT(*) DESC
  7  )
  8  WHERE ROWNUM = 1;

PAYMENT_MODE                 TOTAL
-------------------- ----------
UPI                           4
```

Report 2: Revenue Analysis

Find:

- Total revenue generated

```
SQL> SELECT SUM(Transaction_Amount) AS TOTAL_REVENUE FROM Payment WHERE Payment_Status = 'Success';

TOTAL_REVENUE
-------------
       281800
```

- Revenue by payment method

```
SQL> SELECT Payment_Mode, SUM(Transaction_Amount) AS TOTAL_REVENUE FROM Payment WHERE Payment_Status = 'Success' GROUP BY Payment_Mode;

PAYMENT_MODE        TOTAL_REVENUE
------------------- -------------
Card                       145000
Cash                         6800
UPI                        130000
```

- Average transaction amount

```
SQL> SELECT AVG(Transaction_Amount) AS AVERAGE_TRANSACTION_AMOUNT FROM Payment;

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                47144.4444
```

Report 3: Customer payment history

Find

- Customer Name
- Order ID
- Payment mode
- Amount
- Payment status

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, Orders.ORDER_ID, Payment.PAYMENT_MODE, Payment.TRANSACTION_AMOUNT,
Payment.PAYMENT_STATUS
  2  FROM Customer
  3  JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  4  JOIN Payment ON Orders.ORDER_ID = Payment.ORDER_ID;

FIRST_NAME           LAST_NAME              ORDER_ID PAYMENT_MODE
-------------------- -------------------- ---------- --------------------
TRANSACTION_AMOUNT PAYMENT_STATUS
------------------ --------------------
Beema                ponting                    1001 UPI
            70000 Success

priya                sakthi                     1002 Card
            45000 Success

Arun                 kumar                      1003 Cash
             3400 Success


FIRST_NAME           LAST_NAME              ORDER_ID PAYMENT_MODE
-------------------- -------------------- ---------- --------------------
TRANSACTION_AMOUNT PAYMENT_STATUS
------------------ --------------------
Anu                  Priya                      1004 UPI
            75000 Pending

Beema                ponting                    1006 Card
            55000 Success
```

```
FIRST_NAME           LAST_NAME              ORDER_ID PAYMENT_MODE
-------------------- -------------------- ---------- --------------------
TRANSACTION_AMOUNT PAYMENT_STATUS
------------------ --------------------
Anu                  Priya                      1004 UPI
            75000 Pending

Beema                ponting                    1006 Card
            55000 Success

priya                sakthi                     1007 UPI
            67500 Pending


FIRST_NAME           LAST_NAME              ORDER_ID PAYMENT_MODE
-------------------- -------------------- ---------- --------------------
TRANSACTION_AMOUNT PAYMENT_STATUS
------------------ --------------------
Arun                 kumar                      1008 Card
            45000 Success

Anu                  Priya                      1009 Cash
             3400 Success

Reena                das                        1011 UPI
            60000 Success


9 rows selected.
```

Payment analysis :

```
SQL> SELECT Payment_Mode,
  2  COUNT(*) AS TRANSACTION_COUNT,
  3  SUM(Transaction_Amount) AS TOTAL_REVENUE,
  4  ROUND(AVG(Transaction_Amount), 2) AS AVG_TRANSACTION
  5  FROM Payment
  6  WHERE Payment_Status = 'Success'
  7  GROUP BY Payment_Mode
  8  ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE         TRANSACTION_COUNT TOTAL_REVENUE AVG_TRANSACTION
-------------------- ----------------- ------------- ---------------
Card                                 3        145000        48333.33
UPI                                  2        130000           65000
Cash                                 2          6800            3400

SQL>
```